

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Build a Complete AWS Infrastructure Using Terraform

Scenario

Your company wants to deploy a new application consisting of:

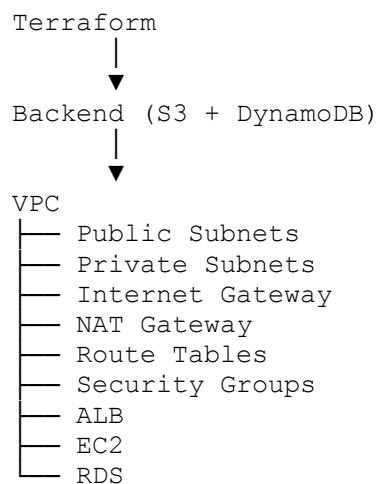
- VPC
- Public & Private Subnets
- Internet Gateway
- NAT Gateway
- EC2
- ALB
- RDS

The infrastructure should be repeatable across Dev, QA, and Production.

How would you design it?

Solution

Architecture



Folder Structure

```
terraform/
├── backend.tf
```

```
├── providers.tf
├── variables.tf
├── terraform.tfvars
├── outputs.tf
├── main.tf
├── modules/
│   ├── vpc
│   ├── ec2
│   ├── alb
│   ├── rds
│   └── security-group
```

Best Practices

- Reusable modules
- Separate environments
- Remote backend
- Version control
- Least privilege IAM

Scenario 2: Two Engineers Run Terraform Apply at the Same Time

Scenario

Two DevOps engineers execute:

```
terraform apply
```

at the same time.

What problems can occur?

How do you prevent them?

Solution

Problem:

- State corruption
- Duplicate resources
- Race conditions

Solution:

Use

S3 Backend

+

DynamoDB State Locking

Flow

Engineer A

↓

Lock Acquired

↓

Apply

↓

Unlock

Engineer B

↓

Wait

Interview Tip

Always mention:

Terraform itself does not support concurrent state modifications. State locking prevents corruption.

Scenario 3: Production Terraform State Was Accidentally Deleted

Scenario

Someone deleted the Terraform state file stored in S3.

How would you recover?

Solution

If S3 Versioning is Enabled

Restore the previous version.

If Versioning is Disabled

Recover using:

- Latest backup
- `terraform import`
- Cloud resource discovery

Prevention

- Enable S3 Versioning
 - Restrict bucket deletion
 - Enable MFA Delete (where appropriate)
 - Regular backups
-

Scenario 4: Create Reusable Terraform Modules

Scenario

Five applications require the same VPC architecture.

How would you avoid duplicate Terraform code?

Solution

Create modules.

Example:

```
modules/
```

```
vpc/
```

```
ec2/
```

```
alb/
```

```
security-group/
```

rds/

Usage

```
module "vpc" {  
  source = "../modules/vpc"  
  
  cidr = "10.0.0.0/16"  
}
```

Benefits

- Reusability
- Maintainability
- Consistency
- Easier upgrades

Scenario 5: Deploy Infrastructure to Multiple AWS Accounts

Scenario

Infrastructure must be deployed to:

- Dev
- QA
- UAT
- Production

Each in a separate AWS account.

How would you manage this?

Solution

Use:

- Separate AWS accounts
- IAM AssumeRole
- Multiple provider configurations

Example

```
provider "aws" {  
  alias = "prod"  
  
  region = "ap-south-1"  
  
  assume_role {  
    role_arn = "arn:aws:iam::123456789012:role/TerraformRole"  
  }  
}
```

Advantages

- Better security
- Account isolation
- Centralized deployments

Scenario 6: Terraform Apply Failed After Creating Half the Resources

Scenario

During deployment:

```
terraform apply
```

fails after creating:

- VPC
- ALB

but EC2 creation fails.

What would you do?

Solution

Step 1

Check state

```
terraform state list
```

Step 2

Identify failed resources.

Step 3

Fix the issue.

Examples:

- IAM permission
- Invalid AMI
- Quota exceeded

Step 4

Run

```
terraform apply
```

again.

Terraform creates only the missing resources because the state already tracks successful ones.

Scenario 7: Existing AWS Infrastructure Must Be Managed by Terraform

Scenario

The company already has:

- VPC
- EC2
- ALB

created manually.

Management wants Terraform to manage them.

Solution

Use

```
terraform import
```

Example

```
terraform import aws_instance.web i-123456789
```

Then

```
terraform plan
```

to reconcile configuration and state.

Benefits

- No recreation
- No downtime
- Infrastructure becomes manageable with Terraform

Scenario 8: Securely Manage Database Passwords

Scenario

Developers store RDS passwords inside:

```
terraform.tfvars
```

Is this a good practice?

Solution

No.

Use:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- CI/CD secret management

Example

```
Terraform
```

↓

```
Secrets Manager
```


↓

RDS

Avoid committing secrets to Git.

Mark outputs containing secrets as:

```
sensitive = true
```

Scenario 9: Terraform Shows Changes Even Though Nothing Was Modified

Scenario

Every execution shows:

```
terraform plan
```

with changes.

Why?

Solution

Possible reasons:

- Manual AWS changes (configuration drift)
- Provider updates
- Computed attributes
- Dynamic timestamps
- Unordered collections

Investigation

```
terraform plan
```

Review differences.

Resolution

- Remove manual changes
- Use `ignore_changes` only when appropriate
- Keep provider versions pinned

Scenario 10: Zero-Downtime Infrastructure Changes

Scenario

Production EC2 instances need replacement.

Users must not experience downtime.

Solution

Use

```
create_before_destroy = true
```

inside the lifecycle block.

Example

```
lifecycle {  
    create_before_destroy = true  
}
```

Flow

New EC2

↓

Health Check

↓

Traffic Shift

↓

Delete Old EC2

Benefits

- Zero downtime
 - Safe replacement
 - Easier rollback
-

Scenario 11: Integrate Terraform with Jenkins CI/CD

Scenario

Every Git push should automatically provision infrastructure.

How would you implement this?

Solution

Pipeline

GitHub

↓

Jenkins

↓

Terraform Init

↓

Terraform Validate

↓

Terraform Plan

↓

Approval (Production)

↓

Terraform Apply

Best Practices

- Store state remotely
 - Protect production with manual approval
 - Archive plan files
 - Use service roles instead of access keys
-

Scenario 12: Manage Multiple Environments

Scenario

The same infrastructure should be deployed to:

- Dev
- Test
- Production

with different configurations.

Solution

Preferred approaches:

Option 1 (Recommended)

Separate directories:

```
environments/
```

```
dev/
```

```
test/
```

```
prod/
```

Each environment has its own:

- Backend
- Variables
- State

Option 2

Terraform Workspaces (better suited to simpler use cases).

Benefits

- Environment isolation
 - Separate state files
 - Reduced risk
-

Scenario 13: Terraform Destroy Is Accidentally Executed in Production

Scenario

Someone runs:

```
terraform destroy
```

against the production environment.

How do you protect critical resources?

Solution

Use lifecycle protection.

Example

```
lifecycle {  
  prevent_destroy = true  
}
```

Also:

- Restrict IAM permissions
 - Require pipeline approvals
 - Protect production branches
 - Review plans before apply
-

Scenario 14: Terraform Deployment Fails Due to Dependency Issues

Scenario

Terraform attempts to create an EC2 instance before the Internet Gateway and routes are ready.

How would you fix this?

Solution

Terraform usually detects dependencies automatically through resource references.

If required, use:

```
depends_on = [  
    aws_internet_gateway.main  
]
```

Interview Tip

Prefer implicit dependencies first.

Use `depends_on` only when Terraform cannot infer the relationship.

Scenario 15: Production Deployment Failed Because Someone Modified AWS Resources Manually

Scenario

A DevOps engineer manually changed the ALB listener through the AWS Console.

The next Terraform deployment failed unexpectedly.

How would you handle this?

Solution

Step 1

Identify drift.

```
terraform plan
```

Step 2

Determine whether the manual change should be kept.

Step 3

Choose one approach:

- Update Terraform code to match the desired configuration.
- Revert the manual change.
- Import additional resources if needed.

Step 4

Apply the corrected configuration.

Prevention

- Enforce Infrastructure as Code.
- Restrict console changes.
- Use change management processes.
- Periodically detect drift.

Terraform on AWS Interview Cheat Sheet

Component	Purpose
Provider	Connects Terraform to AWS
Resource	Creates AWS resources
Data Source	Reads existing AWS resources
Module	Reusable infrastructure component
Variable	Accepts configurable input values
Output	Exposes values after deployment
Local	Stores reusable expressions
State File	Tracks deployed infrastructure
Remote Backend	Stores state remotely (for example, Amazon S3)
DynamoDB Locking	Prevents concurrent state updates
Terraform Import	Brings existing resources under Terraform management
Terraform Plan	Shows proposed infrastructure changes
Terraform Apply	Creates or updates infrastructure
Terraform Destroy	Removes managed infrastructure
Lifecycle	Controls resource creation, replacement, and deletion behavior
Depends_on	Explicit dependency declaration
Sensitive Variables	Protects secret values from output

Production Best Practices

- Store Terraform state in **Amazon S3** with **versioning enabled**.
- Use **DynamoDB** for state locking.

- Organize infrastructure into **reusable modules**.
- Pin Terraform and provider versions.
- Use **IAM roles** instead of long-lived AWS access keys.
- Keep secrets in **AWS Secrets Manager** or **SSM Parameter Store**, not in Git.
- Run `terraform fmt`, `terraform validate`, and `terraform plan` in CI before `apply`.
- Require **manual approval** for production deployments.
- Enable **drift detection** and avoid manual infrastructure changes.
- Protect critical resources using `prevent_destroy` where appropriate.